JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class StreamHandler

java.lang.Object
    java.util.logging.Handler
        java.util.logging.StreamHandler

Direct Known Subclasses:

ConsoleHandler, FileHandler, SocketHandler

```
public class StreamHandler
extends Handler
```

Stream based logging `Handler`.

This is primarily intended as a base class or support class to be used in implementing other logging `Handlers`.

`LogRecords` are published to a given `java.io.OutputStream`.

Configuration: By default each `StreamHandler` is initialized using the following `LogManager` configuration properties where `<handler-name>` refers to the fully-qualified class name of the handler. If properties are not defined (or have invalid values) then the specified default values are used.

- <handler-name>.level specifies the default level for the `Handler` (defaults to `Level.INFO`).
- <handler-name>.filter specifies the name of a `Filter` class to use (defaults to no `Filter`).
- <handler-name>.formatter specifies the name of a `Formatter` class to use (defaults to `java.util.logging.SimpleFormatter`).
- <handler-name>.encoding the name of the character set encoding to use (defaults to the default platform encoding).

For example, the properties for `StreamHandler` would be:

- java.util.logging.StreamHandler.level=INFO
- java.util.logging.StreamHandler.formatter=java.util.logging.SimpleFormatter

For a custom handler, e.g. com.foo.MyHandler, the properties would be:

- com.foo.MyHandler.level=INFO
- com.foo.MyHandler.formatter=java.util.logging.SimpleFormatter

Since:

1.4

Constructor Summary

Constructors

Constructor	Description
`StreamHandler()`	Create a `StreamHandler`, with no current output stream.
`StreamHandler(OutputStream out, Formatter formatter)`	Create a `StreamHandler` with a given `Formatter` and output stream.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`void`	`close()`	Close the current output stream.
`void`	`flush()`	Flush any buffered messages.
`boolean`	`isLoggable(LogRecord record)`	Check if this `Handler` would actually log a given `LogRecord`.
`void`	`publish(LogRecord record)`	Format and publish a `LogRecord`.
`void`	`setEncoding(String encoding)`	Set (or change) the character encoding used by this `Handler`.
`protected void`	`setOutputStream(OutputStream out)`	Change the output stream.

Methods declared in class Handler

`getEncoding`, `getErrorManager`, `getFilter`, `getFormatter`, `getLevel`, `reportError`, `setErrorManager`, `setFilter`, `setFormatter`, `setLevel`

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

StreamHandler

```
public StreamHandler()
```

Create a `StreamHandler`, with no current output stream.

StreamHandler

```
public StreamHandler(OutputStream out,
                     Formatter formatter)
```

Create a `StreamHandler` with a given `Formatter` and output stream.

Parameters:

`out` - the target output stream

`formatter` - Formatter to be used to format output

Method Details

setOutputStream

```
protected void setOutputStream(OutputStream out)
```

Change the output stream.

If there is a current output stream then the `Formatter`'s tail string is written and the stream is flushed and closed. Then the output stream is replaced with the new output stream.

Parameters:

`out` - New output stream. May not be null.

setEncoding

```
public void setEncoding(String encoding)
                 throws UnsupportedEncodingException
```

Set (or change) the character encoding used by this `Handler`.

The encoding should be set before any `LogRecords` are written to the `Handler`.

Overrides:

`setEncoding` in class `Handler`

Parameters:

`encoding` - The name of a supported character encoding. May be null, to indicate the default platform encoding.

Throws:

`UnsupportedEncodingException` - if the named encoding is not supported.

publish

```
public void publish(LogRecord record)
```

Format and publish a `LogRecord`.

The `StreamHandler` first checks if there is an `OutputStream` and if the given `LogRecord` has at least the required log level. If not it silently returns. If so, it calls any associated `Filter` to check if the record should be published. If so, it calls its `Formatter` to format the record and then writes the result to the current output stream.

If this is the first `LogRecord` to be written to a given `OutputStream`, the `Formatter`'s "head" string is written to the stream before the `LogRecord` is written.

Specified by:

`publish` in class `Handler`

Implementation Requirements:

This method avoids acquiring locks during `LogRecord` formatting, but `this` instance is synchronized when writing to the output stream. To avoid deadlock risk, subclasses must not hold locks while calling `super.publish()`. Specifically, subclasses must not define the overridden `publish()` method to be `synchronized` if they call `super.publish()`.

Parameters:

`record` - description of the log event. A null record is silently ignored and is not published

isLoggable

```
public boolean isLoggable(LogRecord record)
```

Check if this `Handler` would actually log a given `LogRecord`.

This method checks if the `LogRecord` has an appropriate level and whether it satisfies any `Filter`. It will also return false if no output stream has been assigned yet or the LogRecord is null.

Overrides:

`isLoggable` in class `Handler`

Parameters:

`record` - a `LogRecord` (may be null).

Returns:

true if the `LogRecord` would be logged.

flush

```
public void flush()
```

Flush any buffered messages.

Specified by:

`flush` in class `Handler`

close

```
public void close()
```

Close the current output stream.

The `Formatter`'s "tail" string is written to the stream before it is closed. In addition, if the `Formatter`'s "head" string has not yet been written to the stream, it will be written before the "tail" string.

Specified by:

`close` in class `Handler`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.